

Pacing

Rôle de chaque module

Guide pour une personne qui reprend le projet

Version du code : 0.2.0 | septembre 2026

Ce document explique le projet **Pacing** à quelqu'un qui ne l'a jamais ouvert. Il décrit ce que fait l'application, comment une donnée circule, puis le rôle de chaque dossier du code. L'objectif est de savoir *où* modifier quelque chose, et *pourquoi* ce morceau existe.

Table des matières

1	À quoi sert le projet	1
2	Le vocabulaire du projet	1
3	Comment le code est rangé	2
3.1	La règle à respecter	2
3.2	Le trajet d'une donnée	2
4	config — chemins et réglages	3
4.1	pacing/config/paths.py	3
4.2	pacing/config/settings.py	3
5	domain — le langage commun	3
5.1	Normalisation — <code>normalize.py</code>	3
5.2	Libellés de nage — <code>stroke_labels.py</code>	4
5.3	Schémas des données brutes	4
6	ingestion — collecter et nettoyer	4
6.1	Extranat — France	4
6.2	Omega — PDF de résultats	4
6.3	USA Swimming — API	5
6.4	FRM Natation — Maroc	5
7	data — lire les fichiers traités	5
7.1	extranat_loader.py	5
7.2	frmnatation_loader.py	5
7.3	usaswimming_loader.py	5
7.4	machine_workers.py	6
8	analytics — les calculs	6
8.1	corridor_data.py — le couloir	6
8.2	graph_compute.py — les autres graphiques	6
9	rendering — dessiner avec Matplotlib	6
10	grammar — la recette du graphique	6
10.1	spec.py	6
10.2	families.py	7
10.3	render_matplotlib.py	7
10.4	corridor.py	7

11 application — les cas d’usage	7
11.1 <code>app_service.py</code> — la façade	7
11.2 <code>graph_service.py</code> — l’orchestrateur	7
11.3 <code>scope.py</code>	8
11.4 <code>api_core.py</code>	8
11.5 <code>build_corridor_chart.py</code>	8
11.6 <code>prefetch_graphs.py</code>	8
11.7 <code>manual_swimmer_store.py</code>	8
12 api — le serveur HTTP	9
13 ui — les écrans Python	9
13.1 Bureau Flet — <code>ui/desktop/</code>	9
13.2 Web NiceGUI — <code>ui/web/</code>	10
13.3 DearPyGUI — <code>ui/dearpygui/</code>	10
13.4 Widgets — <code>ui/widgets/</code>	10
14 ios — le prototype iPad et Mac	10
15 Où intervenir selon la tâche	11
16 Lancer le projet	11
17 Points d’attention pour la suite	11

1 À quoi sert le projet

Pacing analyse le **rythme de course en natation**. À partir de résultats de compétition, on peut :

- consulter des chronos et des passages intermédiaires (les *splits*) ;
 - voir comment un nageur se situe par rapport au peloton de son âge ;
 - comparer deux nageurs sur la même épreuve ;
 - afficher des graphiques : distributions de temps, effectifs, clubs, vitesses, heatmaps, relais.
- Les données viennent de quatre origines :

Source	Pays	Ce que c’est
Extranat	France	Site de la Fédération française de natation
FRM Natation	Maroc	Résultats déjà extraits de pages HTML
USA Swimming	États-Unis	API officielle du Data Hub américain
Omega Timing	Compétitions chronométrées	PDF « Total Ranking »

Dans l’application, les trois pays proposés à l’utilisateur sont la France (FR), le Maroc (MA) et les États-Unis (US).

2 Le vocabulaire du projet

Ces mots reviennent partout. Les connaître évite de se perdre dans les noms de fichiers.

Pacing La façon dont le nageur répartit son effort pendant la course : plus vite au début, plus régulier, ou plus vite à la fin. On le voit surtout avec la vitesse de chaque split.

Épreuve Une course précise : distance + nage + bassin. En interne, un libellé ressemble à 100 FR LCM : 100 mètres, nage libre, grand bassin (50 m).

Code de nage Les données gardent des codes courts en anglais : FR nage libre, BK dos, BR brasse, FL papillon, IM quatre nages. L'écran les affiche en français (« Nage libre », « Dos », etc.).

Bassin LCM = bassin de 50 m. SCM = bassin de 25 m. La même distance n'a pas les mêmes temps selon le bassin.

Split Temps de passage à une distance intermédiaire (50 m, 100 m, ...). Le Maroc en a très peu : les splits y sont en général vides.

Couloir de performance Le graphique central du projet. En abscisse, l'âge (ou la catégorie d'âge aux États-Unis). En ordonnée, le temps. Des bandes colorées montrent les percentiles du peloton. On superpose dessus la courbe d'un nageur.

Scope Le filtre en cours dans l'interface : nage, distance, bassin, parfois le genre. Tous les graphiques n'utilisent pas les mêmes filtres. Un camembert global ignore la nage ; un couloir exige nage, distance et bassin.

ChartSpec Une *recette* de graphique : les données, les axes, les couches à dessiner, la légende. Ce n'est pas une image. L'API envoie cette recette en JSON. L'application iPad la redessine de son côté.

Performance Une ligne de résultat : un nageur (ou un relais), une épreuve, un temps, un club, une date, parfois des splits.

3 Comment le code est rangé

Le code Python vit dans le paquet `pacing/`. Il est découpé en couches. Chaque couche a un seul métier.

Dossier	Rôle en une phrase
<code>config/</code>	Où sont les fichiers, et quels réglages lire au démarrage.
<code>domain/</code>	Le vocabulaire commun : noms, genres, nages, forme des JSON.
<code>ingestion/</code>	Aller chercher les données sur le web et les nettoyer.
<code>data/</code>	Lire les fichiers déjà nettoyés et en faire des tableaux.
<code>analytics/</code>	Calculer (percentiles, vitesses, agrégats) sans dessiner.
<code>rendering/</code>	Dessiner avec Matplotlib, sans décider du calcul.
<code>grammar/</code>	Décrire un graphique de façon indépendante de l'outil de dessin.
<code>application/</code>	Les cas d'usage : charger, filtrer, construire un graphe.
<code>api/</code>	Le serveur HTTP que le web, DearPyGUI et l'iPad appellent.
<code>ui/</code>	Les écrans : bureau Flet, web NiceGUI, prototype DearPyGUI.

À côté du paquet Python :

- `data/` contient les données (brutes, nettoyées, exports). Ce dossier n'est pas versionné : chaque poste a les siennes.
- `services/` ne contient plus de code. Seulement des secrets locaux : le jeton USA Swimming et un fichier d'état.
- `ios/PacingApp/` est le prototype iPad / Mac en Swift.
- `docs/` rassemble la documentation écrite.

3.1 La règle à respecter

Trois phrases suffisent à ne pas mélanger les couches :

1. Le **calcul** ne dessine pas. Pas de Matplotlib dans `analytics/`.
2. Le **dessin** ne décide pas du métier. Pas de règle « quel percentile, quel nageur » dans `rendering/`.
3. L'**interface** ne va pas scraper le web. Elle demande des données déjà prêtes.

3.2 Le trajet d'une donnée

1. **Collecte.** Un script d'*ingestion/* télécharge des pages, des PDF ou des réponses d'API, et écrit du JSON brut dans *data/raw/*.
2. **Nettoyage.** Un second script de la même source uniformise les noms, les genres, les temps en secondes, les codes de nage. Le résultat va dans *data/processed/*.
3. **Lecture.** Un *loader* de *data/* transforme ces fichiers en un grand tableau pandas : une ligne = une performance.
4. **Calcul.** *analytics/* filtre ce tableau et produit des chiffres (percentiles par âge, vitesses de split, effectifs).
5. **Présentation.** Soit Matplotlib dessine une image (application Flet), soit une recette ChartSpec part en JSON (API, web, iPad).

Après nettoyage, les colonnes se ressemblent d'une source à l'autre : **Meet**, **Event**, **Stroke**, **Distance**, **Course**, **SwimTimeSeconds**, **swimmer**, **splits**. C'est le format commun du projet. **swimmer** reste souvent une liste de dictionnaires, parce qu'une course individuelle a un nageur et un relais en a plusieurs.

4 config — chemins et réglages

Deux fichiers. Rien de métier natation ici : seulement « où » et « combien ».

4.1 *pacing/config/paths.py*

C'est l'unique endroit qui connaît les dossiers du projet. Tout le reste doit importer ces chemins, au lieu de remonter dans l'arborescence avec `Path(__file__).parents[...]`.

- *data/raw/* et *data/processed/* pour chaque source (Extranat, Omega, USA, FRM) ;
- le cache Parquet USA : *data/processed/usaswimming/_parquet_cache/* ;
- les secrets : *services/bearer_token.txt* et *services/state.json* ;
- les exports de préchargement : *data/exports/*.

`ensure_project_imports()` ajoute la racine du dépôt au chemin Python. Utile quand on lance un script à la main, sans avoir installé le paquet.

4.2 *pacing/config/settings.py*

Ce fichier lit des variables d'environnement dont le nom commence par `PACING_`. Il ne calcule aucun graphique.

Ces variables pilotent le **préchargement** de l'application bureau. Au démarrage, Flet peut fabriquer des images à l'avance (couloirs, heatmaps, relais, ...) pour qu'un clic ultérieur soit immédiat. Le démarrage est alors plus long, et la mémoire plus sollicitée.

Exemples :

- `PACING_ENABLE_CORRIDOR_CHART_PREFETCH` : oui ou non. Par défaut, oui. Les couloirs sont préparés au lancement.
- `PACING_CORRIDOR_CHART_PREFETCH_LIMIT` : un nombre. Par défaut, 96. On prépare au plus 96 couloirs, pas la totalité des épreuves.

D'autres limites existent pour les heatmaps, la médiane contre le meilleur temps, les relais, et le cache des nageurs par épreuve. Si la variable n'est pas définie, la valeur écrite dans `PrefetchSettings` est utilisée. On peut couper un préchargement sans modifier le code :

```
PACING_ENABLE_CORRIDOR_CHART_PREFETCH=false pacing-desktop
```

5 domain — le langage commun

Ce dossier ne charge pas de fichier et ne trace pas de courbe. Il fixe la façon de parler des nageurs et des épreuves, pour que la France, le Maroc et les États-Unis soient comparables.

5.1 Normalisation — `normalize.py`

Les noms arrivent sous des formes différentes : accents, majuscules, espaces doubles, « Femme », « Female », « H ». Ce module les ramène à une forme comparable.

- `normalize_text` / `normalize_name` : minuscules, sans accent. « Éléonore DUPONT » devient `eleonore dupont`. Sert à retrouver un nageur.
- `normalize_gender_code` : ramène le genre à F ou M.
- `slugify` : transforme un libellé en identifiant de fichier (`100-fr-1cm`).
- `primary_swimmer_name` : sort le nom du premier nageur, que le champ soit un dictionnaire ou une liste (cas du relais).

5.2 Libellés de nage — `stroke_labels.py`

Les calculs gardent FR, BK, BR, FL, IM. L’affichage passe par ce fichier.

- `stroke_code_to_label` : FR devient « Nage libre ».
- `format_event_label` : construit « 100 Nage libre LCM ».
- `localize_event_string` : traduit le code au milieu d’un libellé déjà formé, du type 100 FR LCM.

5.3 Schémas des données brutes

Trois fichiers Pydantic décrivent le JSON *avant* ou *juste autour* du nettoyage. Ils servent à refuser un fichier mal formé plutôt qu’à le laisser casser plus loin.

- `models_extranat.py` — hiérarchie française : compétition, épreuve, performance, nageur. Le champ nageur accepte une personne ou une liste (relais). Les temps sont encore du texte (`00:26.14`).
- `models_frmn.py` — même idée pour le Maroc, avec des noms de champs déjà proches du format unifié (`SwimTime`, `SwimTimeSeconds`). Les splits sont prévus mais presque toujours vides.
- `models_usaswimming.py` — une ligne de « best time » de l’API américaine : nom, épreuve, genre, club, date, temps en secondes, standard de temps.

Si une source ajoute un champ, c’est ici qu’on le déclare. Les modèles ignorent les champs inconnus (`extra = ignore`), pour ne pas casser quand le site ajoute une colonne.

6 ingestion — collecter et nettoyer

Chaque source a son dossier. Le schéma est le même : un script de collecte, puis un script de prétraitement. On les lance à la main, quand on veut rafraîchir les données. L’interface ne les appelle pas.

6.1 Extranat — France

`pacing/ingestion/extranat/service.py` interroge le site `ffn.extranat.fr`.

1. Il lit les types de compétition (national, international, ...).
2. Il parcourt le calendrier, page par page.
3. Il ouvre les résultats : épreuves, classements, clubs, temps, points, meilleure performance personnelle, splits.
4. Il écrit un JSON par compétition dans `data/raw/extranat/`.

Le site protège parfois ses pages (erreurs 403). Le script réessaie avec un délai croissant.

`preprocessing.py` convertit ce JSON brut vers le format unifié : temps en secondes, codes de nage, genre normalisé. La sortie va dans `data/processed/extranat/competitions_per_type/`.

```
python -m pacing.ingestion.extranat.service
python -m pacing.ingestion.extranat.preprocessing
```

6.2 Omega — PDF de résultats

`pacing/ingestion/omega/service.py` parcourt `omegatiming.com` année par année, trouve les compétitions de natation et télécharge les PDF « Total Ranking » dans `data/raw/omega/pdfs/`.

Le site demande des cookies de navigateur. S'ils sont expirés, Playwright ouvre Chromium pour les renouveler (`cookies_omegatiming.txt` à la racine du projet).

`pdf_to_json_claude.py` envoie ensuite chaque PDF au modèle Claude (clé API Anthropic) pour en extraire un JSON structuré. Ce n'est pas un parseur de tableau écrit à la main : c'est une extraction par modèle de langage, avec reprise des PDF déjà traités pour ne pas repayer deux fois le même fichier.

```
python -m pacing.ingestion.omega.service
```

6.3 USA Swimming — API

`pacing/ingestion/usaswimming/service.py` appelle l'API du Data Hub. Il faut un jeton d'accès.

- `get_token.py` ouvre un navigateur (Playwright) pour obtenir ce jeton et l'enregistrer dans `services/bearer_token.txt`.
- Le service télécharge les résultats par plage de dates, par mois, ou en boucle pour suivre les nouvelles nages.
- Les JSON bruts vont dans `data/raw/usaswimming/`.
- `preprocessing.py` les aligne sur le format commun.

Le volume américain est trop gros pour être relu en JSON à chaque ouverture de l'application. Le cache Parquet (dossier `data/`) prend le relais après ce nettoyage.

```
python -m pacing.ingestion.usaswimming.get_token
python -m pacing.ingestion.usaswimming.service
python -m pacing.ingestion.usaswimming.preprocessing
```

6.4 FRM Natation — Maroc

Il n'y a **pas de scraper** dans ce dépôt. Un outil extérieur dépose déjà un JSON par compétition dans `data/raw/frmnatation/`.

`pacing/ingestion/frmnatation/preprocessing.py` fait seulement le ménage :

- noms (casse, mots de famille) ;
- codes de nage marocains (DOS, PAP, 4N) vers les codes internes ;
- chronos illisibles ou nuls, retirés ;
- catégories d'âge.

La sortie va dans `data/processed/frmnatation/html_results/`.

```
python -m pacing.ingestion.frmnatation.preprocessing
```

7 data — lire les fichiers traités

Les loaders transforment les JSON (ou le Parquet) en un tableau pandas. Une ligne correspond à une performance. L'analyse et l'interface travaillent sur ce tableau, pas sur les fichiers.

7.1 extranat_loader.py

`ExtranatCompetitionsDataLoader.load()` parcourt, en parallèle, tous les JSON du dossier traité Extranat (`competitions_per_type/`). Chaque compétition est aplatie : épreuves, puis performances. Le champ `swimmer` et les `splits` sont conservés tels quels.

7.2 `frmnatation_loader.py`

Même idée pour le Maroc, avec un cache en mémoire : les fichiers sont peu nombreux et relus souvent. Le loader sait aussi :

- lister les nageurs pour les menus (« Nom (année) ») ;
- filtrer par épreuve, bassin et genre ;
- préparer un nageur marocain pour le superposer à un couloir français ou américain.

7.3 `usaswimming_loader.py`

`UsaswimmingCompetitionsDataLoader` a deux portes d'entrée.

1. `build_parquet_cache()` lit les JSON annuels et écrit un fichier Parquet par année.
2. `load()` ne lit **que** ce cache. Il ne reparcourt pas les JSON. Les temps de réponse restent prévisibles.

La colonne `swimmer` (liste de dictionnaires) est stockée en texte JSON dans le Parquet, parce que ce format ne garde pas les listes imbriquées. Elle est reconstruite à la lecture.

7.4 `machine_workers.py`

Choisit le nombre de threads selon la RAM et le processeur de la machine (profil modeste, standard, ou puissant). La construction du cache Parquet d'une grosse année peut saturer un ordinateur portable si on lance trop de workers. Ce fichier évite ça.

8 analytics — les calculs

Aucun import de Matplotlib ici. Ces fonctions reçoivent un tableau et en renvoient un autre, ou des séries de nombres. Deux fichiers portent presque tout le métier « chiffres ».

8.1 `corridor_data.py` — le couloir

C'est le cœur du couloir de performance.

- Préparer un tableau « long » : une ligne par point (nageur, âge, temps) pour une épreuve.
- Calculer les percentiles (par exemple 10, 25, 50, 75, 90) et les déciles, selon l'âge.
- Retrouver un nageur par son nom, avec ou sans année de naissance. La recherche est souple : accents et casse sont ignorés.
- Exclure le nageur cible du peloton quand on trace sa courbe par-dessus, pour ne pas le compter deux fois.
- Lire les splits, convertir un passage en vitesse (m/s), et décider si une distance a assez de passages pour un profil de pacing.
- Borner l'axe des âges et expliquer pourquoi un nageur est introuvable.

Les couleurs du nageur français et du nageur marocain sont aussi définies ici, pour que tous les écrans utilisent les mêmes.

8.2 `graph_compute.py` — les autres graphiques

Préparations et agrégats pour le reste du catalogue : ordre des nages (libre, dos, brasse, papillon, quatre nages), effectifs par sexe, bins d'histogramme, tableaux longs pour les heatmaps de vitesse, détection des relais, libellés d'épreuve triés par distance. Le dessin correspondant est dans `rendering/`.

9 rendering — dessiner avec Matplotlib

Ce dossier reçoit des tableaux déjà calculés et produit une figure.

- `chart_plots.py` : histogrammes, barres classées, boxplots, camemberts, évolution des chronos, heatmaps de vitesse, thème visuel commun (couleurs, ticks de temps au format `m:ss`).

- `corridor_plots.py` : bandes de percentiles, bandes de déciles, courbe du nageur, profil de pacing normalisé.

Si une règle du type « quel nageur garder » ou « quel percentile calculer » apparaît dans ces fichiers, elle est au mauvais endroit : elle doit remonter dans `analytics/`.

10 grammar — la recette du graphique

Matplotlib sait dessiner, Swift aussi, mais ils ne parlent pas le même langage. `grammar/` est le langage commun.

10.1 spec.py

`ChartSpec` décrit un graphique complet :

- un type (`corridor`, `histogram`, ...) et un titre ;
- des tables de données nommées ;
- des échelles X et Y ;
- des couches dans l'ordre du dessin : aire, ligne, point, barre, histogramme, boxplot, heatmap, ligne de repère ;
- un thème, une légende, des annotations ;
- éventuellement plusieurs panneaux (petits graphiques côte à côte).

`to_dict()` transforme cette recette en JSON. C'est ce JSON que l'API envoie à l'iPad : le graphique est décrit, il n'est pas envoyé comme image.

10.2 families.py

Construit les recettes des familles courantes à partir de données déjà prêtes : histogramme, barres groupées, barres classées, boxplot, lignes, heatmap, profil de pacing, pacing normalisé. Ajouter un nouveau type de graphique « portable » (visible aussi sur iPad) commence souvent ici.

10.3 render_matplotlib.py

Lit une `ChartSpec` et la dessine avec Matplotlib. C'est le pont entre la recette et l'image, pour le bureau et pour le web quand ils passent par la grammaire.

10.4 corridor.py

Répond une seule fois à la question : « à quoi ressemble un couloir de performance ? ». Bandes, médiane, couleurs, légende, axes. Deux entrées appellent la même construction :

- depuis le JSON de l'API (web, DearPyGUI, iPad) ;
- depuis un tableau de percentiles déjà agrégé (application Flet).

Avant ce fichier, la même définition était recopiée dans le web, dans Matplotlib et dans Swift. Une modification de couleur ou de bande doit se faire ici pour rester cohérente.

11 application — les cas d'usage

Cette couche est celle que l'interface et l'API sont censées appeler. Elle enchaîne chargement, filtre, calcul et rendu. Elle ne scrape pas, et elle ne dessine pas les widgets.

11.1 app_service.py — la façade

`PacingAppService` est le point d'entrée du bureau. Il possède :

- un `ServiceGraphe` ;
- le loader USA et le loader Maroc ;
- le chargement Extranat (mis en cache pour tout le processus) ;
- les listes de nageurs, d'épreuves, et la construction des figures PNG.

Les constantes de pays y sont écrites en français : **France**, **Maroc**, **États-Unis**. L'API les traduit ensuite en **FR**, **MA**, **US**.

Le catalogue des graphiques dépend du pays. Pour les États-Unis, la catégorie « Couloirs de performance » ne propose que le couloir par AgeGroup, parce que les données américaines sont organisées par catégorie d'âge et non par année fine comme en France.

11.2 `graph_service.py` — l'orchestrateur

C'est le plus gros fichier du métier (plusieurs milliers de lignes). **ServiceGraphe** sait construire chaque graphique du menu. Le menu est déclaré dans **DESKTOP_GRAPH_MENU**. Les rubriques actuelles sont :

- distributions de temps (histogramme simple et cumulatif) ;
- effectifs et répartition par sexe ;
- comparaison des temps par nage (boxplot) ;
- clubs (participation, temps médian) ;
- évolution des chronos dans le temps ;
- vitesse globale (par distance, vitesse max de split) ;
- pacing comparatif femmes / hommes, avec nageurs cibles ;
- heatmap de vitesse moyenne (distance × nage) ;
- comparaisons de splits à partir de la médiane (meilleur nageur, top 10, genre) ;
- pacing en relais ;
- couloirs de performance (global, nageur cible, déciles, AgeGroup USA).

Chaque libellé du menu correspond à une méthode. Le service garde aussi des caches en mémoire (splits d'une épreuve, tableau long des heatmaps, splits de relais) pour ne pas recalculer la même épreuve à chaque clic.

Des ensembles comme **SCOPE_NO_FILTER_GRAPHS** disent quels graphiques ignorent le filtre nage / distance / bassin. C'est la liste qui évite d'appliquer un filtre de 100 m nage libre à un camembert « tout le fichier ».

GRAPHES_NOTEBOOK est un second catalogue, plus ancien, avec des clés stables (**Graphe1**, **Graphe2**, ...). Le préchargement au démarrage s'appuie dessus. Le menu visible, lui, s'appuie sur les libellés français. Les deux registres coexistent : en modifier un sans l'autre décale le préchargement et l'écran.

11.3 `scope.py`

Décide du filtre réel d'un graphique.

- **event_combinations** construit l'arbre Nage → Distance → Bassins, pour remplir les menus avec uniquement les combinaisons qui existent dans les données.
- **resolve_scope_filters** dit, pour le graphique choisi, quels filtres appliquer.
- **materialize_df_scope** découpe le grand tableau en conséquence.

11.4 `api_core.py`

Même métier que la façade, mais le résultat est un dictionnaire JSON, sans image. C'est le cœur appelé par FastAPI.

- liste des pays ;
- recherche de nageurs (autocomplétion) ;
- couloir : bandes du peloton + courbe optionnelle d'un nageur ;
- comparaison de deux nageurs sur le même couloir ;
- référentiel des épreuves (nages, distances, bassins) ;
- catalogue des graphiques.

Détail utile : le pays du *peloton* et le pays du *nageur* peuvent différer. On peut afficher un nageur marocain sur un couloir français. Les temps sont en secondes, les âges en années, les distances en mètres.

Le catalogue indique, pour chaque graphique, un champ **endpoint**. S'il est renseigné, le graphique s'obtient en HTTP sous forme de **ChartSpec**. S'il est vide, ce graphique n'existe pour l'instant que dans l'application Flet, en local.

11.5 `build_corridor_chart.py`

Petit cas d'usage : à partir d'un tableau et d'une épreuve, préparer les données du couloir puis demander la figure. Il évite à l'interface de connaître l'enchaînement des fonctions de `corridor_data.py`.

11.6 `prefetch_graphs.py`

Prépare les figures du catalogue « notebook » au démarrage du bureau, dans les limites fixées par `settings.py`.

11.7 `manual_swimmer_store.py`

Enregistre une performance saisie à la main, depuis l'iPad ou le Mac : nom, année de naissance, genre, pays, club, épreuve, temps. Les fichiers vont dans `data/processed/manual_performances/`. Le temps accepté est 63.31 ou 1:03.31. Ces saisies réapparaissent dans la recherche et dans les couloirs, à côté des données scrapées.

12 `api` — le serveur HTTP

FastAPI, préfixe `/api/v1`. Lancement :

```
uvicorn pacing.api.main:app -reload
```

La documentation interactive est sur <http://127.0.0.1:8000/docs>.

Route	Rôle
GET <code>/pays</code>	France, États-Unis, Maroc.
GET <code>/nageur/recherche</code>	Autocomplétion de nageurs dans un pays, éventuellement une épreuve.
POST <code>/nageur</code>	Enregistrer une performance saisie à la main.
GET <code>/couloir</code>	Bandes du peloton, courbe d'un nageur, et la recette ChartSpec .
GET <code>/comparaison</code>	Deux nageurs sur le même couloir.
GET <code>/referentiels/epreuves</code>	Nages, distances et bassins disponibles.
GET <code>/graphiques</code>	Catalogue, avec l'indication de ce qui est disponible en HTTP.

- `main.py` crée l'application et branche le routeur.
- `routers/pacing.py` déclare les routes. Il valide les paramètres, appelle `api_core.py`, puis ajoute la **ChartSpec** sur le couloir et la comparaison.
- `schemas.py` décrit les entrées et sorties avec Pydantic. C'est le contrat : si un champ change ici, les clients (web, iPad) doivent suivre.
- `errors.py` force une seule forme d'erreur : `{"error": {"code", "message"}}`.
- `export_openapi.py` régénère le fichier de schéma OpenAPI. Commande : `python -m pacing.api.export_openapi`. L'option `-check` vérifie que le fichier versionné correspond encore au code.

L'API suppose un usage local ou sur un réseau interne : elle n'a pas d'authentification.

13 `ui` — les écrans Python

Trois interfaces. Deux d'entre elles ne calculent rien : elles appellent l'API. La troisième, Flet, est l'exception.

13.1 Bureau Flet — ui/desktop/

Commande : `pacing-desktop`, ou `python -m pacing.ui.desktop.app`.

C'est l'application la plus complète, et le plus gros fichier du dépôt (`app.py`, plusieurs milliers de lignes). Elle tourne sur l'ordinateur qui possède les données, et elle appelle `PacingAppService` **directement**. Elle ne passe pas par HTTP.

Au démarrage :

1. une barre de progression s'affiche ;
2. les graphiques, le cache des nageurs par épreuve, le Parquet USA et les couloirs sont préparés, selon les réglages `PACING_*` ;
3. la barre latérale (pays, épreuve, nageur, graphique) et la zone d'image apparaissent.

Fichiers à côté :

- `corridor_prefetch.py` précharge les couloirs ;
- `helpers.py` choisit une nage et une distance par défaut, et convertit une figure en image.

Un nouveau graphique du menu Flet se branche dans `DESKTOP_GRAPH_MENU` (`graph_service.py`), avec sa méthode de calcul. On ne recopie pas un second orchestrateur dans `app.py`.

13.2 Web NiceGUI — ui/web/

Commande : `pacing-web`. Port par défaut : 8080. L'API doit déjà tourner (`PACING_API_BASE_URL`, par défaut `http://127.0.0.1:8000`).

Pages : accueil, exploration des données, couloir, comparaison. `api_client.py` fait les appels HTTP. `charts.py` dessine la réponse. Le web ne charge pas les JSON de compétition lui-même.

13.3 DearPyGUI — ui/dearpygui/

Commande : `pacing-dpg`. Prototype de bureau plus léger, lui aussi branché uniquement sur l'API : recherche de nageur, couloir, comparaison, export PNG.

13.4 Widgets — ui/widgets/

Morceaux réutilisés par Flet : barre de progression du préchargement (`loading_progress.py`) et champ de recherche de nageur (`swimmer_search.py`).

14 ios — le prototype iPad et Mac

`ios/PacingApp/` est une application SwiftUI pour un entraîneur sur le terrain. On l'ouvre avec :

```
open ios/PacingApp/PacingApp.xcodeproj
```

Deux modes :

- **Démo**, par défaut : des JSON embarqués dans l'application, sans serveur.
- **Live** : l'application parle à l'API Python. Sur le Mac ou le simulateur, l'adresse est `127.0.0.1`, port 8000. Sur un iPad physique, c'est l'adresse IP de l'ordinateur sur le réseau local. L'API doit alors écouter sur toutes les interfaces (`0.0.0.0`).

Fichiers utiles pour s'orienter :

- `PacingAppApp.swift` : point d'entrée.
- `ContentView.swift` et `Views/` : accueil, recherche, couloir, comparaison, référentiel d'épreuves, réglages.
- `Services/PacingAPIClient.swift` : les appels HTTP, équivalent de `api_client.py`.
- `Services/MockPacingService.swift` : les données de démo.
- `Models/ChartSpec.swift` : la recette de graphique, côté Swift. Elle doit rester alignée avec `pacing/grammar/spec.py`.
- `Views/CorridorChartView.swift` : le dessin du couloir.

Si la forme du JSON de couloir change dans `schemas.py`, il faut mettre à jour ces modèles Swift en même temps. Sinon l'iPad ne saura plus lire la réponse.

15 Où intervenir selon la tâche

Tu veux...	Tu ouvres...
Ajouter ou réparer une collecte	<code>pacing/ingestion/<source>/</code>
Changer la forme du tableau chargé	le loader dans <code>pacing/data/</code>
Changer un percentile, une vitesse, un filtre	<code>pacing/analytics/</code>
Changer l'aspect d'une image Matplotlib	<code>pacing/rendering/</code>
Faire que le web, le bureau et l'iPad dessinent la même chose	<code>pacing/grammar/</code>
Ajouter un graphique au menu Flet	<code>DESKTOP_GRAPH_MENU</code> dans <code>graph_service.py</code> , puis la méthode
Exposer ce graphique à l'iPad ou au web	<code>api_core.py</code> , <code>api/routers/pacing.py</code> , <code>api/schemas.py</code> , puis le client Swift ou NiceGUI
Changer un écran	<code>ui/desktop</code> , <code>ui/web</code> , <code>ui/dearpygui</code> , ou <code>ios/PacingApp</code>
Déplacer un dossier de données	<code>pacing/config/paths.py</code> seulement
Rendre le démarrage du bureau plus léger	variables <code>PACING_*</code> lues par <code>settings.py</code>

16 Lancer le projet

Prérequis : Python 3.10 ou plus, et Chromium via Playwright si on scrape Omega ou si on renouvelle le jeton USA.

```
python -m venv .venv
source .venv/bin/activate
pip install -e .
playwright install chromium
```

Interface	Commande
Bureau complet	<code>pacing-desktop</code>
API	<code>uvicorn pacing.api.main:app -reload</code>
Web	<code>pacing-web</code> (l'API doit tourner)
Bureau léger	<code>pacing-dpg</code> (l'API doit tourner)
iPad / Mac	ouvrir le projet Xcode dans <code>ios/PacingApp/</code>

Sans le dossier `data/processed/`, les écrans s'ouvrent mais les listes d'épreuves et de nageurs sont vides. Les données ne sont pas dans git : il faut les générer avec les scripts d'ingestion, ou les copier depuis un poste qui les a déjà.

17 Points d'attention pour la suite

1. **Deux très gros fichiers** : `graph_service.py` et `ui/desktop/app.py`. On y ajoute un cas dans les structures déjà prévues. On n'y colle pas un second programme à côté.
2. **Deux catalogues de graphiques**. Les libellés du menu Flet et les clés `GRAPHES_NOTEBOOK`. Un graphique visible à l'écran n'est pas automatiquement préchargé, ni automatiquement disponible dans l'API.
3. **L'API ne couvre pas tout le menu Flet**. Le champ `endpoint` du catalogue le dit. Le couloir et la comparaison, eux, passent déjà par l'API et par `ChartSpec`.

4. **Flet n'est pas un client HTTP.** Une correction faite seulement dans `api_core.py` n'atteint pas le bureau Flet, et l'inverse est vrai. Le code partagé est en dessous : `analytics/`, `loaders`, `ServiceGraphe`.
5. **Le format swimmer est irrégulier.** Tantôt un dictionnaire, tantôt une liste. Les loaders et `normalize.py` uniformisent. Un nouveau parseur doit faire de même.
6. **USA = Parquet.** Après un nouveau téléchargement américain, il faut reconstruire le cache (`build_parquet_cache`), sinon l'application continue de lire l'ancien fichier.
7. **Secrets hors du code.** Jeton USA et cookies Omega restent sur le poste, dans `services/` et à la racine. On ne les commit pas.
8. **Swift et Python partagent un contrat.** Toute modification de `pacing/grammar/spec.py` ou des schémas de couloir doit être répercutée dans `ios/PacingApp/`.

En cas de doute sur un changement : identifier la couche (collecte, tableau, calcul, dessin, recette, écran), modifier cette couche, et vérifier l'autre bout de la chaîne si un JSON ou une `ChartSpec` sort du processus Python.